

● Alta — Te afectan directamente al usar el sistema

1. Selects no muestran texto → Cuando creás un producto, pedido, bulto o miembro, los dropdowns de categorías, roles, couriers aparecen vacíos. No ves qué opción seleccionaste. El README ya lo documenta como bug conocido. Es el más frustrante porque cada formulario del admin lo sufre.
2. API usa body crudo → Si definís valores por defecto en Zod (ej: type: "chico" por defecto), esos defaults se ignoran. Podés terminar creando bultos sin tipo o categorías sin nombre porque el schema validó bien pero el código usó los datos sin transformar.
3. costUSDT usa priceUSD como fallback → Si creás un producto sin especificar costo, el sistema asume que el costo ES el precio de venta. Esto te calcula márgenes de ganancia del 0% y puede distorsionar reportes financieros sin que te des cuenta.
4. Register no asigna role → Si te registrás como nuevo admin, probablemente quedás sin rol. Después no podés acceder a secciones protegidas y pensás que el login está roto, cuando en realidad el usuario no tiene permisos.
5. Truthy checks en vez de !== undefined → Si un envío sale \$0 (gratis) o querés poner status vacío, el sistema lo ignora y no actualiza el campo. Cambiás un costo a ARS 0 y el sistema no prorratea porque 0 es falsy.

● Media — Molestan o pueden causar errores silenciosos

6. useEffect sin dependencias → Al editar un producto, si cambiás la categoría, el formulario puede no reaccionar. O al ver un producto, el color seleccionado queda desincronizado si cambiás de producto rápido.
7. Categorías con colores hardcoded → Si usás el tema claro (que Next.js soporta con next-themes), la página de categorías muestra texto blanco sobre fondo blanco. Invisible. Solo funciona en dark mode.
8. Rate limiter sin limpieza → Con uso prolongado (días/semanas), la memoria del server crece con cada IP única que visita. Eventualmente puede ralentizar o crashear el server en Railway. No te afecta hoy, pero sí con tráfico constante.
9. StatusBadge parsea clases → Visualmente menor, pero si reorganizás las clases CSS, los badges de estado (Pendiente/En camino/Entregado) pueden perder el color de texto y verse inconsistentes.
10. Reportes sin campos → La sección Reportes del admin no te deja elegir destino, fechas ni tipo. Solo envía un reporte genérico al admin. No coincide con lo que promete el README ni con lo que necesitás si querés enviar un reporte específico.

● Baja — Riesgo controlado

11. as any → Si la API cambia la forma de los datos (ej: imágenes pasan de string[] a objetos), la app rompe en runtime sin warning previo de TypeScript.
12. fetchExchangeRate → Solo funciona en cliente. Si en el futuro lo usás desde un Server Component, falla sin error claro.
13. AdminLayout sin verificar rol → Un customer (rol de solo lectura) podría acceder al panel. No debería poder.

¿Arrancamos con los 5 rojos? Especialmente el #1 (selects) y #4 (register sin rol) son los que más se sienten al usar la app hoy.

Aquí la lista completa de los 17 archivos modificados:

Bug	Archivo	Cambio
1	components/ui/select.tsx	CSS problemático removido
2	api/categorias/route.ts	usa parsed.data
2	api/bultos/route.ts	usa parsed.data
3	api/productos/[slug]/route.ts	costUSDT fallback 0
4	api/register/route.ts	role: "admin"
5	api/bultos/[id]/route.ts	truthy → !== undefined
6	components/admin/ProductForm.tsx	useEffect deps
6	(public)/productos/[slug]/page.tsx	useEffect + as any
7	(public)/categorias/[slug]/page.tsx	colores theme-aware
8	lib/rate-limit.ts	limpieza cada 5min
9	admin/bultos/page.tsx	StatusBadge textColor
9	admin/importacion/page.tsx	StatusBadge textColor
10	admin/reportes/page.tsx	campos email/fechas/tipo
11	admin/productos/[id]/editar/page.tsx	as removidos
11	admin/finanzas/page.tsx	: any → : OrderOption
12	lib/exchange-rate.ts	guard client-side
13	admin/layout.tsx	verifica rol